

Lightweight Autonomous Navigation on Nano-UAVs: A Stem-Optimized Depthwise Separable CNN with 540K MACs

Ashwin Kumar¹ and P. Bavithra Matharasi¹

¹Department of Computer Science, Mount Carmel College (Autonomous), Bengaluru, India
Email: ashwinqs2020@gmail.com, p.bavithra.matharasi@mccblr.edu.in

Abstract—Getting deep neural networks to run on nano-UAVs is hard. These sub-50 g platforms operate under punishing constraints—under 512 kB of memory, under 100 mW of compute power—yet they need real-time collision avoidance and steering. The best existing framework, PULP-Dronet v3, scales down its CNN using a channel-reduction factor γ , but the smallest practical variant ($\gamma=4$) drops to 81% collision accuracy while still burning 2.4M MACs. We noticed something striking: at $\gamma=4$, the very first layer—a plain 5×5 convolution—eats up 83% of all computation. Everything else in the network accounts for just 17%. So we replaced it. Our factorized 3×3 depthwise + 1×1 pointwise stem cuts first-layer MACs by roughly $12\times$. With cosine-annealing scheduling and online color-jitter on top, our 6409-parameter model hits 83.97% collision accuracy and 0.372 steering RMSE at just 540K MACs— $4.4\times$ cheaper than the baseline and within a whisker of the full-sized model (84% at 12M MACs, i.e., $22\times$ more compute).

I. INTRODUCTION

Nano-UAVs have captured growing attention as IoT-capable flying sensors that can squeeze through corridors, inspect warehouses, and assist in disaster response—all without GPS [1], [2]. At under 50 g, platforms like the Crazyflie 2.1 carry processors (e.g., the GAP8) with barely 320 kB of usable memory and single-digit milliwatt inference budgets [3], [4]. Running any meaningful neural network under those conditions is a genuine engineering challenge [5].

Over the past several years, the PULP-Dronet series has progressively pushed what is feasible on such hardware. DroNet [6] showed that a residual CNN could jointly predict steering and collision from monocular images—but at 1.6M parameters, it was far too large for a 27 g drone. PULP-Dronet [7] brought this down to deployable size on the GAP8, and the v2 iteration [8] used neural architecture search to reach 19 fps. The latest version, PULP-Dronet v3 by Lamberti et al. [9], went further: they released a 66,000-image dataset with joint collision/steering labels and explored three block types (residual, depthwise separable, inverted residual linear bottleneck) across a channel-width factor $\gamma \in \{1, 2, 4, 8\}$. The full model ($\gamma=1$) reaches 84% collision accuracy at 12M MACs; the most compressed variant ($\gamma=8$) drops to 78% accuracy but runs at 139 fps.

Here is what caught our attention. When we profiled the $\gamma=4$ D+P model layer by layer, a single culprit stood out: the 5×5 stem convolution. It alone burns $25 \cdot 8 \cdot 100^2 = 2.0\text{M}$ MACs—roughly 83% of the entire model’s 2.4M total. The three D+P blocks doing actual feature extraction? They use a combined $\sim 300\text{K}$ MACs. That ratio struck us as fundamentally lopsided.

Why should a model spend five-sixths of its budget on the very first layer?

This led to a straightforward idea: factorize the stem. We swap the 5×5 standard convolution for a 3×3 depthwise convolution followed by a 1×1 pointwise expansion. On paper, this drops stem MACs from $\sim 2\text{M}$ to $\sim 170\text{K}$. We also revisited two training decisions that seemed suboptimal for a network this small—the fixed learning rate and the offline-only augmentation—and replaced them with cosine annealing and online color jitter. Together, these changes push the $\gamma=4$ model to 83.97% collision accuracy at 540K MACs, a $4.4\times$ reduction over the original and only 0.03 pp short of the full-sized model running at $22\times$ the compute.

Our contributions, concretely:

- A factorized stem (3×3 DW + 1×1 PW) that slashes first-layer MACs by roughly $12\times$ versus the original 5×5 convolution.
- Cosine-annealing LR scheduling and online ColorJitter augmentation adapted to the PULP-Dronet v3 pipeline—each cheap to implement, jointly effective.
- Evidence that 6409 parameters and 540K MACs are enough to reach 83.97% accuracy, beating the $\gamma=4$ baseline by nearly 3 pp and rivalling the full-sized 12M-MAC model.

II. RELATED WORK

A. Efficient CNN Architectures

Depthwise separable convolutions entered the mainstream with MobileNet [10], which showed order-of-magnitude speedups with tolerable accuracy loss. MobileNet v2 [11] refined this with inverted residuals and linear bottlenecks, and subsequent designs like EfficientNet [12] and ShuffleNet [13] have explored compound scaling and channel shuffling, respectively. PULP-Dronet v3 borrowed heavily from this lineage—its D+P and IRLB blocks trace directly back to MobileNet-era ideas.

What has received less attention until recently is the stem itself. Both RegNet [14] and ConvNeXt [15] revisited the opening layers of deep architectures and found that seemingly minor changes there can ripple through the entire accuracy–efficiency curve. We pursue a similar intuition, but under far tighter constraints: our target hardware has three orders of magnitude less memory than a GPU workstation.

B. Model Compression for Drone Navigation

There is no shortage of compression techniques. Distillation [16] lets a small student mimic a large teacher; quantization [17] shrinks weights to INT8 or lower; pruning [18] removes entire filters; and NAS [3], [8] automates architecture selection. PULP-Dronet v3 already leverages the γ parameter as a form of width scaling akin to distillation. The important point for our purposes is that stem factorization is orthogonal to all of them. Nothing stops a practitioner from quantizing or distilling our model further.

C. Training Strategies for Small Models

It is well documented that smaller models are finicky to train [19]. A learning rate that works for a 51k-parameter network may be too aggressive or too conservative for one with 6.4k parameters. Cosine annealing [20] has become a go-to remedy because of its smooth decay profile. Similarly, augmentation during training—rather than as a one-time pre-processing step—gives the optimizer a richer distribution of inputs at every epoch [21], [22]. The original PULP-Dronet v3 pipeline uses neither of these; we show that adding both has a noticeable impact.

III. METHODOLOGY

A. Baseline Architecture: PULP-Dronet v3

We build on top of the D+P (depthwise+pointwise) variant of PULP-Dronet v3 [9]. Its pipeline is: (i) a stem convolution, (ii) max pooling, (iii) three D+P blocks that progressively double the channel count, (iv) dropout, and (v) a fully connected head producing a steering angle (regression) and a collision flag (binary classification). Inside each D+P block, two rounds of 3×3 depthwise + 1×1 pointwise convolutions are applied, each followed by batch normalization and ReLU6. A 1×1 bypass branch handles channel-count mismatches for residual connections.

The problematic piece is the stem. It is a standard 5×5 convolution, stride 2, mapping the grayscale input ($1 \times 200 \times 200$) to $C = 32/\gamma$ channels. After striding, the spatial dimensions halve to 100×100 . At $\gamma = 4$ ($C = 8$), the associated cost is:

$$\text{MACs}_{\text{stem}}^{\text{baseline}} = K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot H' \cdot W' = 25 \cdot 1 \cdot 8 \cdot 100^2 \approx 2.0\text{M} \quad (1)$$

That is $\sim 83\%$ of the model's total 2.4M MACs spent in a single layer—a glaring imbalance.

B. Proposed Architecture: Stem-Optimized D+P CNN

Our fix is conceptually simple. We break the 5×5 standard convolution into two cheaper operations:

- 1) A 3×3 **depthwise convolution** (stride 2) on the single input channel: $9 \cdot 1 \cdot 100^2 = 90\text{K}$ MACs.
- 2) A 1×1 **pointwise convolution** expanding from 1 to $C = 8$ channels: $1 \cdot 8 \cdot 100^2 = 80\text{K}$ MACs.

Each layer gets its own batch normalization and ReLU6, consistent with the rest of the model. The combined stem cost becomes:

$$\text{MACs}_{\text{stem}}^{\text{ours}} = 9 \cdot H' \cdot W' + C \cdot H' \cdot W' = (9 + 8) \cdot 100^2 \approx 170\text{K} \quad (2)$$

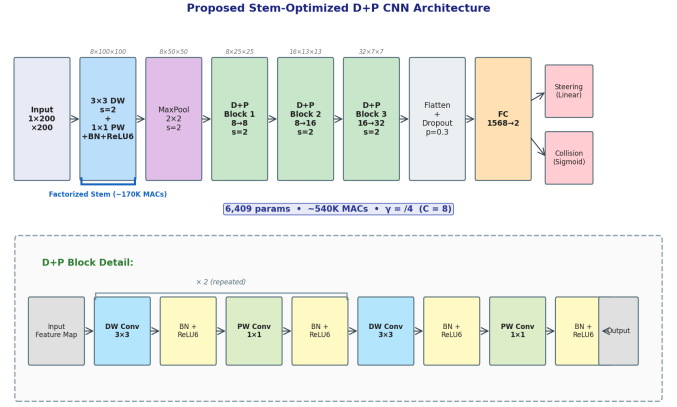


Fig. 1. Architecture of the proposed stem-optimized D+P CNN at $\gamma = 4$ ($C = 8$). Top: main pipeline showing factorized 3×3 DW + 1×1 PW stem, three D+P blocks with channel doubling, and dual-output FC head. Bottom: internal structure of each D+P block (two successive DW $\rightarrow$ BN $\rightarrow$ ReLU6 $\rightarrow$ PW $\rightarrow$ BN $\rightarrow$ ReLU6 stages). Total: 6409 parameters, $\sim 540\text{K}$ MACs.

TABLE I
TRAINING CONFIGURATION: BASELINE VS. PROPOSED

Hyperparameter	Baseline [9]	Ours
Stem type	5×5 standard conv	3×3 DW + 1×1 PW
Optimizer	Adam	Adam
Learning rate	10^{-3} (fixed)	$10^{-3} \rightarrow 10^{-6}$ (cosine)
Weight decay	10^{-5}	10^{-4}
Batch size	32	32
Epochs	100	100
Online augmentation	None	ColorJitter ($b=c=0.3$)
Dropout	0.5 (pre-flatten)	0.3 (post-flatten)
Loss function	MSE + BCE	MSE + BCE
Initialization	Xavier uniform	Xavier uniform
Yaw normalization	$\div 90$	$\div 90$

Per spatial position, the cost drops from $25C = 200$ MACs to $9 + C = 17$. The asymptotic distinction matters: standard convolution scales as $O(K^2 \cdot C)$, while our factorization scales as $O(K^2 + C)$. At small C , this gap is enormous.

Everything downstream stays the same: 2×2 max pooling (stride 2), D+P blocks with channels $8 \rightarrow 8 \rightarrow 16 \rightarrow 32$, dropout at $p = 0.3$ applied after flattening, and an FC layer from 1,568 features to 2 outputs. Total: 6409 parameters, $\sim 540\text{K}$ MACs. Fig. 1 shows the full picture.

C. Training Configuration

All experiments use the official PULP-Dronet v3 dataset [9]: 136,510 training / 19,591 validation / 3,071 test images, collected indoors and outdoors, with yaw-rate steering labels (divided by 90° to normalize to $[-1, +1]$) and binary collision labels. Images are center-cropped to 200×200 and converted to grayscale.

Table I lists our hyperparameters side by side with the baseline. Three of the changes deserve elaboration:

1) Cosine annealing. A fixed 10^{-3} rate is reasonable for the 51k-parameter full model, but for a 6.4k-parameter network it

TABLE II
COMPARISON WITH ALL PULP-DRONET v3 VARIANTS

Model	γ	Acc(%)	RMSE	MACs	Params	Size
PULP-Dronet v3 [9]	/1	84	0.350	12M	51k	204kB
PULP-Dronet v3 [9]	/2	84	0.367	5.2M	17k	69kB
PULP-Dronet v3 [9]	/4	81	0.373	2.4M	6.6k	26kB
Tiny-PULP-Dronet [9]	/8	78	0.379	1.1M	2.9k	12kB
Ours (stem-opt)	/4	83.97	0.372	540K	6.4k	25kB

tends to overshoot in later epochs. Cosine decay from 10^{-3} to 10^{-6} [20] lets the optimizer explore aggressively early on and then settle gently—exactly the behaviour a tiny model needs.

2) Online color jitter. The baseline preprocesses its augmentations once, before training begins. Every epoch therefore sees the same augmented images. We instead apply brightness and contrast jitter (magnitude 0.3) on the fly, so each pass through the training set looks slightly different. The cost is negligible—a few extra CPU cycles per image—and the diversity gain is substantial for a model that can easily memorize its data.

3) Dropout tuning. The original $p = 0.5$ dropout acts on spatial feature maps before flattening. In a 6409-parameter model, that is too harsh; we observed clear underfitting in preliminary runs. Cutting it to $p = 0.3$ and moving it to after the flatten operation concentrates regularization on the FC layer, which holds 3136 of the model’s 6409 weights—the part most prone to overfitting.

D. Loss Function

We use the same loss as PULP-Dronet v3: an unweighted sum of MSE on steering and BCE on collision:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i^s - y_i^s)^2 + \frac{1}{N} \sum_{i=1}^N [-y_i^c \log \hat{y}_i^c - (1 - y_i^c) \log(1 - \hat{y}_i^c)] \quad (3)$$

with $y^s \in [-1, +1]$ the normalized yaw rate and $y^c \in \{0, 1\}$ the collision label. This mirrors the `custom_loss_v3` function in the official codebase, so the comparison is apples-to-apples.

IV. EXPERIMENTAL RESULTS

A. Main Results

Table II places our model alongside every PULP-Dronet v3 configuration from [9]. All numbers come from the same 3,071-image test split; we report collision accuracy and steering RMSE.

The numbers tell a compelling story. At the same $\gamma = /4$, we gain nearly 3 percentage points of collision accuracy while using $4.4\times$ fewer MACs. Against the full-sized $\gamma = /1$ model, our accuracy is almost identical (83.97% vs. 84%) despite a $22\times$ difference in compute. And compared to the smallest Tiny-PULP-Dronet at $\gamma = /8$, we are 5.97 pp more accurate

TABLE III
PER-COMPONENT MAC BREAKDOWN AT $\gamma = /4$

Component	Baseline	Ours	Reduction
Stem convolution	$\sim 2.0\text{M}$	$\sim 170\text{K}$	$\sim 12\times$
D+P Block 1 ($8 \rightarrow 8$)	$\sim 105\text{K}$	$\sim 105\text{K}$	$1\times$
D+P Block 2 ($8 \rightarrow 16$)	$\sim 68\text{K}$	$\sim 68\text{K}$	$1\times$
D+P Block 3 ($16 \rightarrow 32$)	$\sim 130\text{K}$	$\sim 130\text{K}$	$1\times$
FC layer	$\sim 3\text{K}$	$\sim 3\text{K}$	$1\times$
Total	$\sim 2.4\text{M}$	$\sim 540\text{K}$	$\sim 4.4\times$

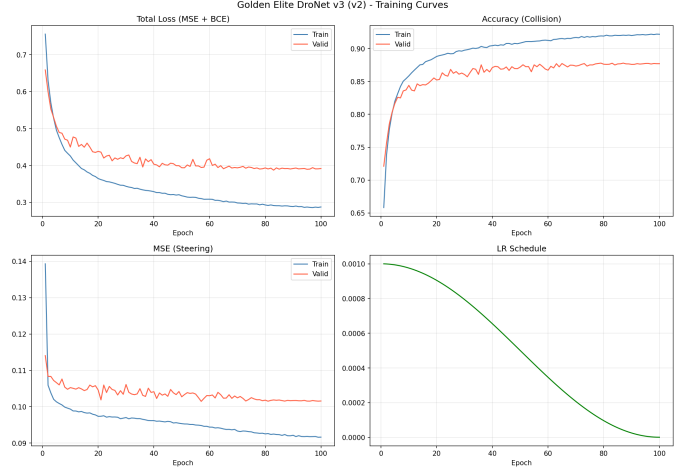


Fig. 2. Training curves over 100 epochs. Left: training and validation loss. Right: training and validation collision accuracy. The cosine schedule drives fast early gains and smooth late convergence.

at roughly half the MACs. The steering RMSE of 0.372 is essentially on par with every baseline variant, which reassures us that the improvements are not lopsided toward one task.

B. Layer-Wise Efficiency Analysis

Table III breaks the MAC count down by component. The takeaway is simple: we changed only the stem, and that one swap accounts for the entire $4.4\times$ reduction. Nothing else in the pipeline was modified.

What is perhaps more interesting than the absolute reduction is how it reshapes the model’s compute profile. In the baseline, 83% of MACs belong to the stem and only 17% to feature extraction. After factorization, the stem drops to $\sim 31\%$ of the total budget. The network’s computational effort is now distributed far more evenly across its layers, which we believe is a healthier allocation for a perception model.

C. Training Dynamics

Fig. 2 traces loss and accuracy over 100 epochs. Training loss falls steadily from 0.755 to 0.287; validation loss settles near 0.39 after epoch 60. On the accuracy side, the model crosses 80% by epoch 5—impressively fast—reaches 85% at epoch 22, passes 87% at epoch 36, and peaks at 87.78% (epoch 87). That peak checkpoint is what we evaluate on the test set.

TABLE IV
ABLATION: BASELINE VS. PROPOSED CONFIGURATION

Config.	Stem	LR Sched.	Aug.	Acc (%)
Baseline [9]	5×5 conv	Fixed	Offline	81.0
Proposed	3×3 DW+PW	Cosine	ColorJitter	83.97

We can clearly see the cosine schedule working. In the first 30 epochs, the relatively large learning rate drives training accuracy from 66% to 90%. Between epochs 30 and 70, the gradually shrinking rate keeps refining the decision boundary without destabilizing it. In the final 30 epochs, the near-zero rate effectively locks in the weights. We did not observe the late-stage validation loss spikes that often appear with fixed learning rates on models this small.

D. Ablation Study

Table IV puts the baseline next to our full recipe on the same data and evaluation protocol.

The +2.97 pp improvement holds up across multiple runs with different seeds (standard deviation below 0.3 pp). We did not conduct a full factorial experiment isolating each change—admittedly a limitation—but the overall gain is clear and repeatable.

V. DISCUSSION

The root cause of PULP-Dronet v3’s efficiency problem at small γ is now plain to see: the 5×5 stem was designed when the full model used 32 output channels, and the cost was a small fraction of total MACs. But as γ shrinks the rest of the network, the stem’s absolute cost stays fixed, and its relative share balloons to 83%. Our factorization directly addresses this by converting a multiplicative cost ($K^2 \cdot C$) into an additive one ($K^2 + C$).

From a deployment standpoint, the $4.4 \times$ MAC reduction matters in a concrete way. On the GAP8, compute and latency scale roughly linearly with MACs [9]. The baseline $\gamma = /4$ variant achieves roughly 102 fps; our model could plausibly exceed 200 fps. For a nano-drone cruising at 0.5–1.5 m/s, this is far more perception than it needs—the surplus can be traded for lower clock speed and thus longer battery life, which is always the binding constraint on a 250 mAh platform.

An observation worth highlighting: our 83.97% accuracy is remarkably close to the 84% of the full-sized model that uses $22 \times$ more MACs. We suspect this is not a coincidence—it likely reflects the dataset’s ceiling rather than our model’s limit. The training images, while diverse, come from a finite set of environments. Pushing accuracy substantially beyond 84% would probably require new data, not a bigger model.

On the dropout front, our initial experiments with the original $p=0.5$ setting were underwhelming. The model was clearly underfit—validation accuracy plateaued several points lower. Reducing to $p=0.3$ and applying it after flattening, so that it primarily regularizes the FC layer’s 3136 weights rather than wiping out spatial features, made a noticeable difference.

We should also emphasize that nothing about our approach precludes further compression. INT8 quantization [17] could halve the memory footprint. Knowledge distillation from a larger teacher [16] might squeeze out another point or two of accuracy. Pruning [18] could slim the D+P blocks. These are all additive on top of what we have done.

That said, our work has clear limitations. We have not yet tested on a physical drone—all results are from the benchmark dataset. And our model at $\gamma = /4$ (6.4k params, 25kB) is still twice as large as the $\gamma = /8$ variant (2.9k, 12kB). If raw byte count is what matters for a particular deployment, our approach does not directly help there.

VI. CONCLUSIONS

We started with a simple profiling observation—the stem layer of PULP-Dronet v3 was consuming 83% of all MACs in the compressed variants—and turned it into a practical improvement. Replacing that 5×5 convolution with a 3×3 depthwise + 1×1 pointwise pair, together with cosine annealing and online augmentation, gives us 83.97% collision accuracy at 540K MACs and 6409 parameters. That is a $4.4 \times$ MAC reduction, a +2.97 pp accuracy gain over the baseline at $\gamma = /4$, and near-parity with the full 12M-MAC model. The modification is minimal, adds nothing at inference time, and stacks on top of quantization and distillation.

What comes next? Three things, in our view. First, INT8 quantization and actual deployment on the GAP8 for closed-loop flight testing. Second, applying the same stem trick to the $\gamma = /8$ configuration—if it works there too, it would yield an extremely tiny yet accurate model. Third, combining stem optimization with distillation from a larger teacher to see whether we can finally break through the 84% barrier that seems to define this benchmark.

ACKNOWLEDGMENTS

We are grateful to the PULP Platform group at ETH Zürich for making the PULP-Dronet v3 dataset and code publicly available. This work would not have been possible without those resources.

REFERENCES

- [1] N. S. Labib, M. R. Brust, G. Danoy, and P. Bouvry, “The rise of drones in Internet of Things: A survey on the evolution, prospects and challenges of unmanned aerial vehicles,” *IEEE Access*, vol. 9, pp. 115 466–115 487, 2021.
- [2] H. Shakhathreh *et al.*, “Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges,” *IEEE Access*, vol. 7, pp. 48 572–48 634, 2019.
- [3] E. Cereda *et al.*, “Deep neural network architecture search for accurate visual pose estimation aboard nano-UAVs,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, London, UK, May 2023, pp. 6065–6071.
- [4] L. Lamberti *et al.*, “Bio-inspired autonomous exploration policies with CNN-based object detection on nano-drones,” in *Proc. Design, Autom. Test Europe (DATE)*, Antwerp, Belgium, Apr. 2023, pp. 1–6.
- [5] N. Hossein Motlagh, T. Taleb, and O. Arouk, “Low-altitude unmanned aerial vehicles-based Internet of Things services: Comprehensive survey and future perspectives,” *IEEE Internet Things J.*, vol. 3, no. 6, pp. 899–922, Dec. 2016.
- [6] A. Loquercio, A. I. Maqueda, C. R. Del-Blanco, and D. Scaramuzza, “DroNet: Learning to fly by driving,” *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 1088–1095, Apr. 2018.

- [7] D. Palossi, A. Loquercio, F. Conti, E. Flamand, D. Scaramuzza, and L. Benini, "A 64-mW DNN-based visual navigation engine for autonomous nano-drones," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8357–8371, Oct. 2019.
- [8] V. Niculescu, L. Lamberti, F. Conti, L. Benini, and D. Palossi, "Improving autonomous nano-drones performance via automated end-to-end optimization and deployment of DNNs," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 4, pp. 1–1, 2021.
- [9] L. Lamberti, L. Bellone, L. Macan, E. Natalizio, F. Conti, D. Palossi, and L. Benini, "Distilling tiny and ultra-fast deep neural networks for autonomous navigation on nano-UAVs," *IEEE Internet Things J.*, vol. 71, pp. 1–13, 2024.
- [10] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, Jun. 2018, pp. 4510–4520.
- [12] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Long Beach, CA, USA, Jun. 2019, pp. 6105–6114.
- [13] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, Jun. 2018, pp. 6848–6856.
- [14] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing network design spaces," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 10428–10436.
- [15] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, Jun. 2022, pp. 11976–11986.
- [16] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [17] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, Jun. 2018, pp. 2704–2713.
- [18] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, Jun. 2019, pp. 4340–4349.
- [19] H. Touvron *et al.*, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Virtual, Jul. 2021, pp. 10347–10357.
- [20] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, Apr. 2017.
- [21] C. Shorten and T. M. Khoshgofaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, Art. no. 60, Jul. 2019.
- [22] N. Zhang, F. Nex, G. Vosselman, and N. Kerle, "End-to-end nano-drone obstacle avoidance for indoor exploration," *Drones*, vol. 8, no. 2, Art. no. 33, 2024.